

Ejercicio de Metodologías Ágiles

Trabajo n.º 12: Ejercicios sobre Metodología Lean

Materia: Metodologías Ágiles
Docente: Sartorio Alejandro
Carrera: Ingeniería en Sistemas
Institución: Universidad Abierta Interamericana
Curso: 4 – A Turno Tarde
Alumno: Miguel A. Campilongo

1) CASO 1:

#Contexto;

Un cliente informa que recibe funcionalidades que no pidió o que no se ajustan a sus necesidades.

Tareas:

Crear una tarea "Detectar origen del retrabajo".

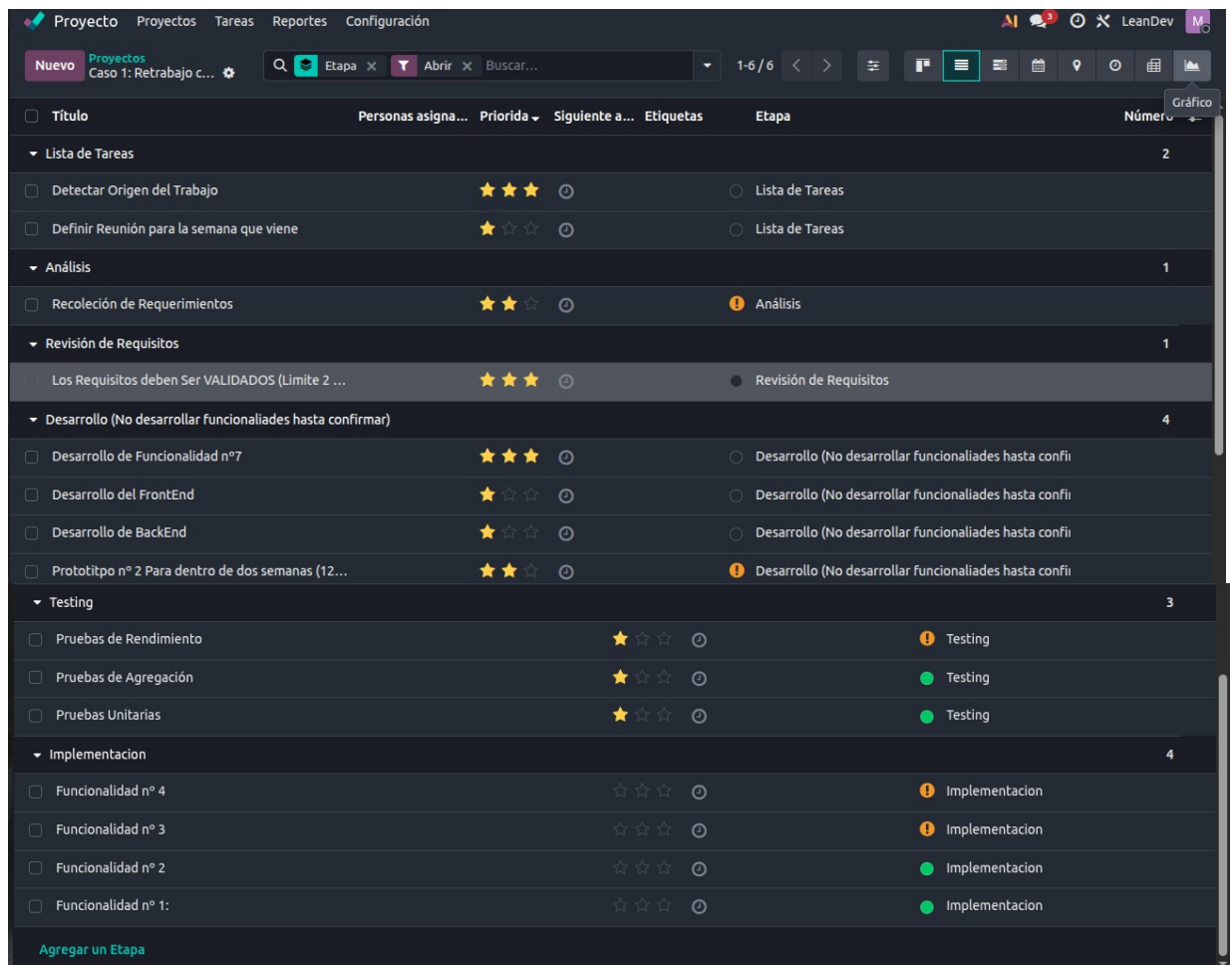
Implementar una mejora en el flujo que garantice la validación temprana de requisitos.

Agregar una etapa "Revisión de Requisitos" en el flujo Kanban.

Limitar el WIP en validaciones.

#Resolución:

Panel del Kanban:



The screenshot shows a Kanban board with the following structure:

Título	Personas asigna...	Priorida	Siguiente a...	Etiquetas	Etapa	Número
Lista de Tareas						2
☐ Detectar Origen del Trabajo		★★★★	○		○ Lista de Tareas	
☐ Definir Reunión para la semana que viene		★☆☆	○		○ Lista de Tareas	
Análisis						1
☐ Recolección de Requerimientos		★★★★	○		! Análisis	
Revisión de Requisitos						1
Los Requisitos deben Ser VALIDADOS (Límite 2 ...		★★★★	○		● Revisión de Requisitos	
Desarrollo (No desarrollar funcionalidades hasta confirmar)						4
☐ Desarrollo de Funcionalidad nº7		★★★★	○		○ Desarrollo (No desarrollar funcionalidades hasta confi	
☐ Desarrollo del FrontEnd		★☆☆	○		○ Desarrollo (No desarrollar funcionalidades hasta confi	
☐ Desarrollo de BackEnd		★☆☆	○		○ Desarrollo (No desarrollar funcionalidades hasta confi	
☐ Prototipo nº 2 Para dentro de dos semanas (12...		★★★★	○		! Desarrollo (No desarrollar funcionalidades hasta confi	
Testing						3
☐ Pruebas de Rendimiento		★☆☆	○		! Testing	
☐ Pruebas de Agregación		★☆☆	○		● Testing	
☐ Pruebas Unitarias		★☆☆	○		● Testing	
Implementacion						4
☐ Funcionalidad nº 4		☆☆☆	○		! Implementacion	
☐ Funcionalidad nº 3		☆☆☆	○		! Implementacion	
☐ Funcionalidad nº 2		☆☆☆	○		● Implementacion	
☐ Funcionalidad nº 1:		☆☆☆	○		● Implementacion	

At the bottom, there is a button labeled "Agregar un Etapa".

Explicación:

Durante las últimas iteraciones el cliente informó que estaba recibiendo funcionalidades que no había solicitado o que no se ajustaban correctamente a sus necesidades. Esto generó retrabajo, pérdida de tiempo y desperdicio de recursos del equipo de desarrollo.

Ante esta situación, el equipo realizó una reunión de mejora del proceso y decidió implementar cambios en el flujo Kanban con el objetivo de reducir errores y validar los requerimientos antes de continuar desarrollando funcionalidades.

Como primera medida, se creó una nueva etapa denominada “Revisión de Requisitos”, ubicada entre “Análisis” y “Desarrollo”. En esta etapa, los requerimientos y funcionalidades deben ser confirmados y validados junto a los usuarios antes de continuar con el desarrollo.

Además, se definió una política de trabajo para evitar continuar codificando funcionalidades incorrectas:

- Las funcionalidades desarrolladas deben ser revisadas y validadas antes de seguir avanzando sobre ellas.
- Ninguna tarea puede pasar completamente a Desarrollo sin aprobación previa.
- Se establece un límite WIP de 2 tareas en la etapa “Revisión de Requisitos” para evitar sobrecarga y mantener un flujo de trabajo controlado.

Con estas medidas el equipo busca:

- Reducir el retrabajo.
- Eliminar desperdicios de desarrollo innecesario.
- Mejorar la comunicación con los usuarios.
- Incrementar la calidad de las funcionalidades entregadas.
- Detectar errores de interpretación de manera temprana.

La implementación de estas políticas permite visualizar mejor los cuellos de botella, controlar el flujo de trabajo y aplicar principios Lean y Kanban orientados a la mejora continua.

2) CASO

Bloqueo en testing por sobrecarga:

Múltiples tareas esperan validación en Testing, sin suficiente capacidad.

Tareas:

Detectar cuello de botella.

Redefinir WIP limit en "Testing".

Crear política de "No nuevas tareas si Testing está lleno".

Documentar impacto esperado.

#) Resolución:

Panel Kanban:

☐ Título	Personas asignadas	Prioridad	Siguiente actividad	Etiquetas	Etapas	Número
☒ Tareas a Hacer						3
☐ Preparar Prototipo para Cliente		★★★★	🕒		⚠️ Tareas a Hacer	
☐ Reunión 27/5: Confirmar proxima iteracion		★☆☆☆	🕒		○ Tareas a Hacer	
☐ Notificar nueva política: No agregar mas tareas si Testing esta lleno		☆☆☆☆	🕒		○ Tareas a Hacer	
☒ Analisis						1

Nombre	Personas asignadas	Progreso	Seguimiento de tareas	Etiquetas	Equipo	Numero
▼ Analisis						1
☐ Recolección de Requerimientos (0/2 subtareas)	★ ★ ☆	🕒		● Analisis		
▼ Diseño						1
☐ Preparar todo para cuando esten los últimos requerimientos	★ ☆ ☆	🕒		○ Diseño		
▼ Desarrollo						2
☐ Funcionalidad 7	★ ☆ ☆	🕒		● Desarrollo		
☐ Funcionalidad 6	★ ☆ ☆	🕒		● Desarrollo		
▼ Testin (WIP = 2)						5
☐ Funcionalidad 5	★ ★ ☆	🕒		○ Testin (WIP = 2)		
☐ Funcionalidad 4	★ ★ ☆	🕒		○ Testin (WIP = 2)		
☐ Funcionalidad 3	★ ★ ☆	🕒		○ Testin (WIP = 2)		
☐ Funcionalidad 2	★ ★ ☆	🕒		○ Testin (WIP = 2)		
☐ Funcionalidad 1	★ ★ ☆	🕒		○ Testin (WIP = 2)		

Explicación:

Durante el último ciclo de trabajo el equipo logró avanzar correctamente en las tareas de desarrollo, pero hacia el final de la iteración el área de Testing informó que estaba recibiendo más tareas de las que podía validar. Se detectó entonces un cuello de botella en la etapa de Testing, ya que el flujo de trabajo del equipo de Desarrollo avanzaba más rápido que la capacidad de validación del equipo de pruebas.

Ante esta situación, el equipo decidió implementar nuevas políticas dentro del tablero Kanban con el objetivo de controlar el flujo de trabajo y evitar la sobrecarga del sector de Testing.

Como primera medida, se redefinió el límite WIP de la etapa “Testing”, estableciendo un máximo de 2 tareas simultáneas en validación. De esta manera se busca evitar acumulación excesiva de trabajo y mantener un ritmo sostenible para el equipo encargado de las pruebas.

Además, se implementó una política de trabajo:

- “No ingresar nuevas tareas a Testing si la etapa alcanzó su límite WIP”.

Esto implica que el equipo de Desarrollo no podrá continuar enviando funcionalidades a Testing hasta que exista capacidad disponible en dicha etapa. Gracias a esta medida, el cuello de botella queda claramente visible dentro del tablero y el equipo puede tomar decisiones para equilibrar el flujo de trabajo.

Se espera que esta política genere un impacto positivo en el proceso de trabajo, ya que:

- Reducirá la sobrecarga del equipo de Testing.
- Evitará acumulación excesiva de tareas pendientes de validación.
- Mejorará la calidad de las pruebas realizadas.
- Permitirá mantener un flujo de trabajo más equilibrado y sostenible.
- Concientizará al equipo sobre la importancia de respetar la capacidad de trabajo de cada etapa del proceso.
- Hará visibles los cuellos de botella para facilitar futuras mejoras continuas.

3) CASO 3

Muchos errores son detectados después de la liberación.

Tareas:

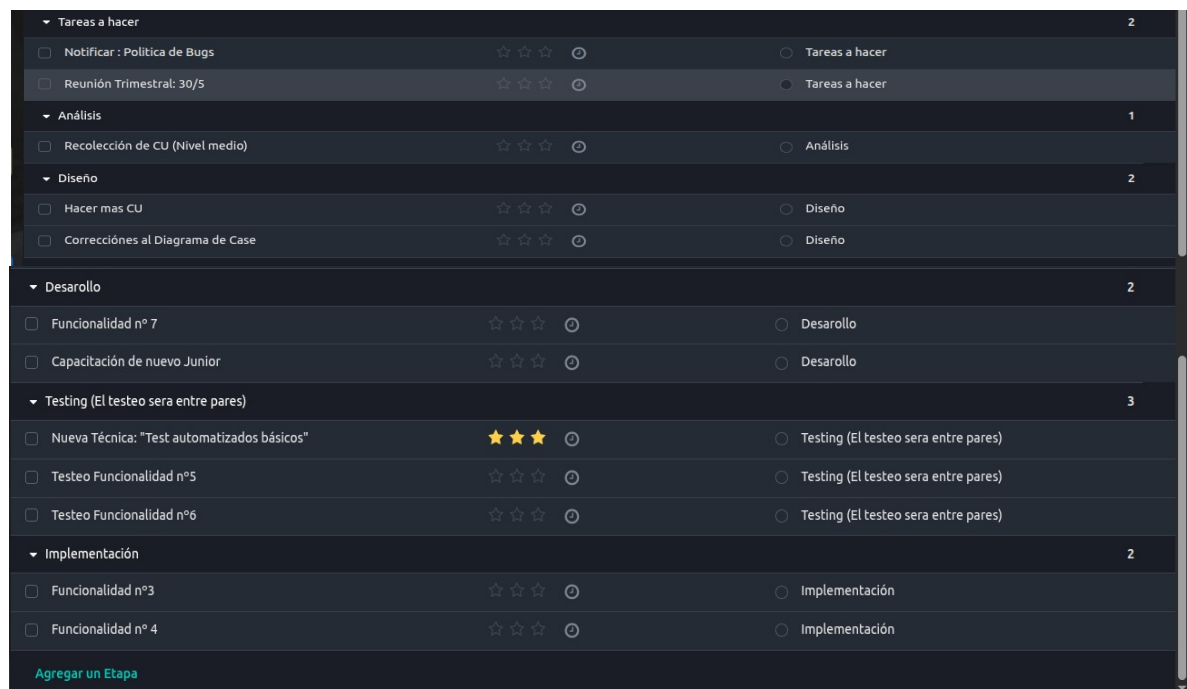
Implementar revisiones entre pares.

Crear tareas de "Tests automatizados básicos".

Etiquetar como prevención de errores.

#Resolución

Panel Kanban:



Explicación:

Durante los últimos días, el equipo de Implementación notificó que las funcionalidades provenientes del área de Testing presentaban una cantidad de errores y bugs mayor a la esperada. Esta situación generaba inconvenientes al momento de poner en marcha las funcionalidades, además de retrasos en el trabajo del equipo, ya que debían invertir tiempo adicional corrigiendo fallas que no habían sido detectadas en etapas anteriores del proceso.

A partir de este problema, el equipo decidió implementar nuevas prácticas orientadas a la prevención de errores y a la mejora de la calidad del software antes de su liberación.

Las medidas adoptadas fueron las siguientes:

- Implementar revisiones entre pares:
Las tareas de Testing deberán ser revisadas por más de una persona antes de considerarse finalizadas. La revisión colaborativa permite detectar errores con mayor facilidad, compartir

conocimientos entre los integrantes del equipo y encontrar soluciones más eficientes a los bugs detectados.

- Crear tareas de “Tests automatizados básicos”:
Además de las revisiones manuales, se incorporarán pruebas automatizadas básicas para validar funcionalidades de manera rápida y repetitiva. Estos tests permitirán realizar múltiples verificaciones en pocos minutos, aumentando la cobertura de pruebas y reduciendo la posibilidad de errores no detectados.
- Etiquetar las tareas como “Prevención de errores”:
Las actividades relacionadas con revisiones colaborativas y pruebas automatizadas serán identificadas dentro del tablero Kanban con la etiqueta “Prevención de errores”, permitiendo visualizar iniciativas orientadas a mejorar la calidad del producto y reducir retrabajos futuros.

Se espera que estas nuevas prácticas permitan entregar funcionalidades con una mayor calidad y estabilidad antes de llegar a Implementación. Gracias a las revisiones entre pares y a los tests automatizados, el equipo podrá detectar errores de forma temprana, disminuyendo la cantidad de bugs presentes en producción y de esta forma la deuda técnica

4) CASO 4

El cliente no utiliza varias funcionalidades entregadas.

Tareas:

Identificar funcionalidades no usadas.

Planificar entregas incrementales.

Crear tareas “Demo para el cliente” antes de cerrar features

#Resolución

Panel Kanban:

▼ Tareas a hacer	2
☐ Identificar funcionalidades no usadas por el cliente	☆☆☆ ⦿ Tareas a hacer
☐ Reunion 29/5	☆☆☆ ⦿ Tareas a hacer
▼ Analisis	1
☐ Recolección de requerimientos	☆☆☆ ⦿ Analisis
▼ Diseño	1
☐ Correccion del Diagrama de Robustez	☆☆☆ ⦿ Diseño
▼ Desarrollo	3
☐ Prueba con el Cliente	☆☆☆ ⦿ Desarrollo
☐ Funcionalidad 7	☆☆☆ ⦿ Desarrollo
☐ Funcionalidad 8	☆☆☆ ⦿ Desarrollo
▼ Testeo	1
☐ Funcionalidad 6	☆☆☆ ⦿ Testeo
▼ Implementación	2
☐ Funcionalidad 5	☆☆☆ ⦿ Implementación
☐ Funcionalidad 4	☆☆☆ ⦿ Implementación
Agregar un Etapa	

Explicación:

Durante las últimas entregas, el equipo detectó que varias funcionalidades desarrolladas no estaban siendo utilizadas por el cliente una vez implementadas. Esto generó preocupación, ya que se estaban invirtiendo tiempo y recursos en desarrollar características que no aportaban valor real al usuario final.

Luego de analizar la situación, el equipo concluyó que algunas funcionalidades habían sido desarrolladas sin suficiente validación continua con el cliente, lo que provocó diferencias entre lo que el equipo consideraba útil y las verdaderas necesidades del usuario.

Para reducir este desperdicio y mejorar la alineación con el cliente, se decidieron implementar las siguientes medidas:

- Identificar funcionalidades no utilizadas:
El equipo realizará un análisis de las funcionalidades entregadas para detectar cuáles no están siendo utilizadas o no generan valor para el cliente. Esta información permitirá comprender mejor las necesidades reales del usuario y evitar desarrollos innecesarios en el futuro.
- Planificar entregas incrementales:
En lugar de desarrollar grandes conjuntos de funcionalidades antes de recibir feedback, se adoptará una estrategia de entregas pequeñas e incrementales. De esta manera, el cliente podrá probar y validar funcionalidades de forma temprana, permitiendo realizar ajustes antes de continuar desarrollando nuevas características.
- Crear tareas "Demo con cliente" antes de cerrar features:
Antes de considerar una funcionalidad como finalizada, el equipo deberá realizar una demostración con el cliente para validar que cumple correctamente con sus necesidades y expectativas. Estas tareas serán incorporadas dentro del flujo Kanban para asegurar que la validación forme parte oficial del proceso.

Se espera que estas medidas permitan mejorar la comunicación y la colaboración con el cliente, asegurando que las funcionalidades desarrolladas realmente aporten valor al producto.

5) CASO 5

Programadores seniors saturados; juniors ociosos.

Tareas:

Balancear la asignación de tareas.

Crear tareas "Mentoría rápida".

Planificar sesiones de apoyo.

#Resolucion

Panel Kanban:

☐ Título	Personas asignadas	Prioridad	Siguiente actividad	Etiquetas	Etapas	Número
▼ Tareas a hacer						2
☐ Planificar sesiones de apoyo para Desarrollo		☆☆☆	🕒		○ Tareas a hacer	
☐ Organizar reunion 27/5: Division de Trabajo		☆☆☆	🕒		○ Tareas a hacer	
▼ Análisis						1
☐ Realizar entrevistas con Usuario para resolver dudas		☆☆☆	🕒		○ Análisis	
▼ Diseño						1
☐ Merorar Diagrama de Secuencia		☆☆☆	🕒		○ Diseño	
▼ Desarrollo (Seniors/Juniors)						3
☐ Mentoría rápida para Juniors		☆☆☆	🕒		○ Desarrollo (Seniors/Juniors)	
☐ Funcionalidad nº 13: Seniors		☆☆☆	🕒		○ Desarrollo (Seniors/Juniors)	
☐ Funcionalidad nº 11: Juniors		☆☆☆	🕒		○ Desarrollo (Seniors/Juniors)	
▼ Testeo						2
☐ Funcionalidad nº 10		☆☆☆	🕒		○ Testeo	
☐ Funcionalidad nº 9		☆☆☆	🕒		○ Testeo	
▼ Implementación						1
☐ Funcionalidad nº 8		☆☆☆	🕒		○ Implementación	

Explicación:

Durante las últimas iteraciones el equipo detectó una distribución desigual del trabajo entre los desarrolladores. Los programadores seniors se encontraban saturados de tareas críticas y consultas técnicas, mientras que varios programadores juniors permanecían con poca carga de trabajo o realizando tareas de baja complejidad.

Esta situación comenzó a generar retrasos en el desarrollo, dependencia excesiva de ciertos integrantes y un desaprovechamiento de la capacidad del equipo. Además, los desarrolladores seniors tenían menos tiempo disponible para enfocarse en tareas estratégicas debido a la cantidad de interrupciones y asistencia técnica constante.

Para mejorar el equilibrio del trabajo y fomentar el crecimiento del equipo, se decidieron implementar las siguientes medidas:

- **Balancear la asignación de tareas:**
El equipo reorganizará la distribución de tareas para evitar la sobrecarga de los programadores seniors y aprovechar mejor la capacidad de los juniors. Las tareas serán asignadas considerando complejidad, experiencia y oportunidades de aprendizaje, permitiendo una distribución más equilibrada del trabajo.
- **Crear tareas de “Mentoría rápida”:**
Se incorporarán tareas específicas de apoyo técnico y mentoría breve dentro del tablero Kanban. Estas actividades permitirán que los seniors puedan ayudar a los juniors de forma organizada y planificada, evitando interrupciones constantes y facilitando la transferencia de conocimientos.
- **Planificar sesiones de apoyo:**
El equipo organizará sesiones periódicas de acompañamiento y resolución de dudas para los desarrolladores juniors. Estas reuniones permitirán fortalecer conocimientos técnicos, mejorar la autonomía del equipo y reducir la dependencia permanente de los integrantes más experimentados.

Se espera que estas medidas permitan mejorar la colaboración y el equilibrio de trabajo dentro del equipo, reduciendo la saturación de los desarrolladores seniors y aprovechando mejor la capacidad disponible de los juniors.